TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN
CÔNG NGHỆ PHẦN MỀM

BrewLite

Ứng dụng đặt cà phê không dùng tiền mặt
Triển khai 10 task theo Agile Scrum

Thành viên	MSSV	Phân công
Trần Bảo Tín	3124410356	Task 1, 6
Nguyễn Thanh Phú	3123410270	Task 2, 9
Vĩnh Nguyễn Phước Bảo Thiện	3124410339	Task 3, 5
Dư Vĩnh Lợi	3123410203	Task 4, 7
Trần Vĩnh Thuận	3124410350	Task 8, 10

GVHD: Từ Lãng Phiêu
Học kỳ I – Năm học 2026–2027
TP. Hồ Chí Minh, tháng 10 năm 2026

Mục lục

1 Tóm tắt và phạm vi **2**

2 Yêu cầu và kiến trúc **2**

2.1 Bên liên quan . . . 2
2.2 Yêu cầu chức năng và phi chức năng . . . 2
2.3 Kiến trúc và công nghệ . . . 3
2.4 Luồng đặt món và thanh toán . . . 3

3 Quy trình Agile Scrum **4**

4 Phân công và commit **5**

5 Minh chứng cho 10 task **5**

5.1 Task 1 — Khởi tạo dự án và cấu trúc . . . 6
5.2 Task 2 — API danh sách sản phẩm . . . 6
5.3 Task 3 — Trang Menu . . . 8
5.4 Task 4 — Chi tiết và tùy chọn sản phẩm . . . 9
5.5 Task 5 — Giỏ hàng . . . 10
5.6 Task 6 — API tạo đơn hàng . . . 10
5.7 Task 7 — Đăng ký và đăng nhập bằng JWT . . . 11
5.8 Task 8 — Thanh toán không tiền mặt . . . 12
5.9 Task 9 — Xác nhận, lịch sử đơn và bàn giao . . . 13
5.10 Task 10 — Nghiệp vụ backend . . . 14

6 Kiểm thử, Definition of Done và bàn giao **17**

6.1 Kết quả kiểm thử ngày 21/10/2026 . . . 17
6.2 Definition of Done . . . 18
6.3 Kịch bản demo bàn giao . . . 18
6.4 Sprint Retrospective . . . 18

7 Nguồn và cách chạy lại **19**

1 Tóm tắt và phạm vi

BrewLite cho khách chọn đồ uống, chọn size và topping, quản lý giỏ hàng, đăng nhập, tạo đơn, thanh toán bằng Ví hoặc Thẻ qua cổng giả lập và xem lịch sử đơn. Frontend viết bằng Next.js, backend bằng NestJS, dữ liệu lưu trong PostgreSQL qua Prisma. Docker Compose dựng ba dịch vụ: cơ sở dữ liệu ở cổng 5432, backend ở cổng 3000, frontend ở cổng 3001.

Đề chia sản phẩm thành 10 task, mỗi task 1 điểm. Báo cáo đối chiếu từng task với tiêu chí ở mục 8 và mục 9 của đề. Số liệu kiểm thử lấy từ lần chạy ngày 21/10/2026 trên nhánh `develop-agile`.

2 Yêu cầu và kiến trúc

2.1 Bên liên quan

Bên liên quan	Việc làm được trên BrewLite	Kênh
Khách hàng	Xem menu, chọn size và topping, đặt đơn, thanh toán, hủy đơn của mình, xem lịch sử và điểm thưởng	Web
Nhân viên quầy (Barista)	Xem hàng đợi đơn đã thanh toán, chuyển đơn sang các bước pha chế, hủy đơn đã thanh toán khi hết nguyên liệu	REST API, kiểm bằng Postman
Quản lý (Admin)	Mọi việc của Barista, xem kho kể cả món hết hàng, chỉnh tồn kho	REST API, kiểm bằng Postman
Cổng thanh toán giả lập	Chấp nhận hoặc từ chối giao dịch Ví/Thẻ	Backend gọi trực tiếp

Web không có màn hình cho Barista và Admin. Mục 3.2 của đề xếp chức năng quầy ngoài MVP, nên nhóm chỉ làm phần API phục vụ máy trạng thái đơn hàng của Task 10.

2.2 Yêu cầu chức năng và phi chức năng

STT	Yêu cầu chức năng (mục 4.1 của đề)	Task
1	Hiển thị menu đồ uống kèm hình, tên, giá	Task 2, 3
2	Xem chi tiết sản phẩm, chọn size và topping	Task 4
3	Thêm, sửa, xóa sản phẩm trong giỏ; tự tính tổng tiền	Task 5
4	Đăng ký, đăng nhập bằng JWT	Task 7
5	Tạo đơn và thanh toán không tiền mặt	Task 6, 8
6	Hiển thị xác nhận đơn và lịch sử đơn	Task 9

Yêu cầu phi chức năng (mục 4.2 của đề)	Minh chứng
API phản hồi dưới 500 ms với dữ liệu mẫu	Lần chạy Postman ngày 21/10/2026: trung bình 14 ms, lớn nhất 108 ms
Băm mật khẩu, JWT, validate đầu vào	bcrypt trong `backend/src/auth/auth.service.ts`; `jwt-auth.guard.ts` chặn request thiếu token với mã 401; DTO dùng class-validator trả 400
Dùng được trên các trình duyệt web	[Cần bổ sung: trình duyệt và phiên bản đã thử]

2.3 Kiến trúc và công nghệ

Thành phần	Công nghệ
Frontend	Next.js, TypeScript; giỏ hàng lưu bằng Zustand
Backend	NestJS, REST, class-validator, JWT
Cơ sở dữ liệu	PostgreSQL qua Prisma, đúng 5 thực thể Product, User, Order, OrderItem, Payment
Thanh toán	Cổng giả lập `backend/src/payment/mock-payment-gateway.service.ts`
Bàn giao	Docker Compose, README, collection Postman `docs/postman/BrewLite.postman_collection.json`

Backend tính lại giá từ bảng giá ở `backend/src/order/pricing.ts` và bỏ qua giá client gửi lên, nên khách không sửa được số tiền phải trả.

2.4 Luồng đặt món và thanh toán

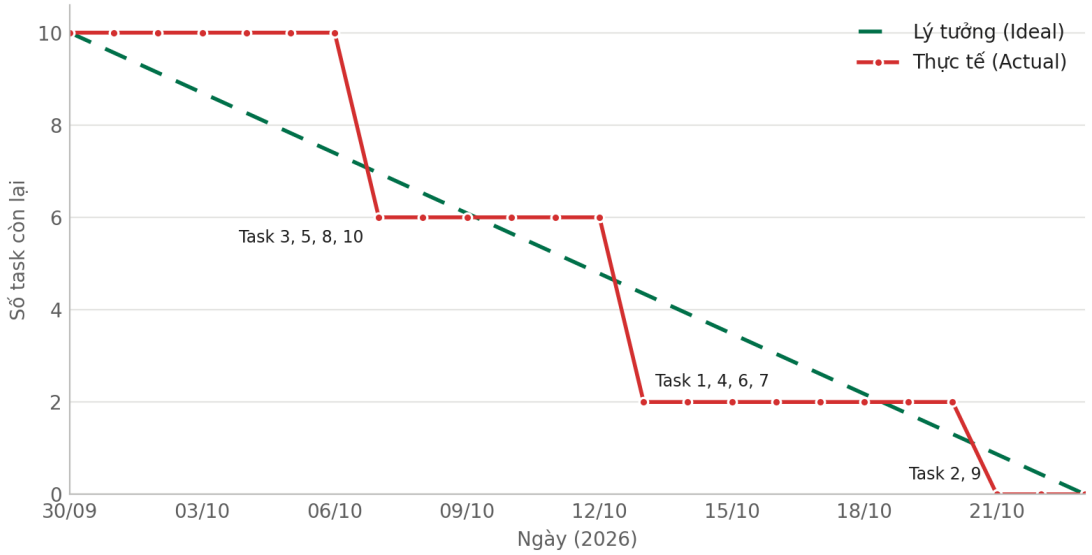
1. Khách mở menu, chọn món, size và topping rồi thêm vào giỏ.
2. Khách đăng nhập nếu chưa có phiên, rồi bấm thanh toán.
3. Frontend gọi `POST /orders` để tạo đơn ở trạng thái Chờ thanh toán và nhận mã đơn.
4. Frontend gọi `POST /payments` với mã đơn, phương thức Ví hoặc Thẻ và khóa chống lặp `Idempotency-Key`, tức chuỗi ngẫu nhiên client gửi trong header để backend nhận ra request lặp lại.
5. Cổng chấp nhận thì đơn sang Đã thanh toán, web hiện biên lai. Cổng từ chối thì đơn sang Thanh toán lỗi, giỏ giữ nguyên và khách thử lại trên chính đơn đó.

3 Quy trình Agile Scrum

Sprint	Mục tiêu (mục 2.3 của đề)	Task
1	Nền tảng và hiển thị menu	1, 2, 3, 4
2	Giỏ hàng, đặt đơn, đăng nhập	5, 6, 7
3	Thanh toán, bàn giao, nghiệp vụ nâng cao	8, 9, 10

Product Backlog gồm 10 user story ở `docs/product-backlog.md`, mỗi story ứng với một task của đề. Một task chỉ tính xong khi đạt Definition of Done ở mục 10 của đề.

Burndown dưới đây tính cho cả 10 task. Một task tính là xong vào ngày tác giả tạo commit của thành viên phụ trách, theo bảng ở mục 4. Đường lý tưởng giảm đều từ 10 task ngày 30/09/2026, ngày có commit đầu tiên của repo, về 0 ngày 23/10/2026, ngày Review 1.



Hình 1: Burndown 10 task dựng từ ngày commit, sinh bằng `burndown.py`

Ngày	Task có commit	Commit	Thực tế	Lý tưởng
30/09	Không có, repo được khởi tạo	`0d022c0`	10	10
07/10	Task 3, 5, 8, 10	`9ea0933`, `bd1adc3`	6	7,0
13/10	Task 1, 4, 6, 7	`44e3784`, `32831dc`	2	4,3
21/10	Task 2, 9	`a358930`, `ed2e4ee`	0	0,9
23/10	Không có, ngày Review 1	Không có	0	0

Những ngày không có trong bảng giữ nguyên số task của ngày liền trước. Theo cách đọc ở mục 2.4 của đề, đường thực tế nằm trên đường lý tưởng là chậm tiến độ. Nhóm chậm trong ba khoảng: 01/10 đến 06/10 khi chưa task nào có commit, 10/10 đến 12/10 trước đợt

commit ngày 13/10, và 19/10 đến 20/10 khi Task 2 và Task 9 chưa có commit. Các ngày còn lại nhóm đi trước đường lý tưởng. Không có ngày bắt đầu và kết thúc từng Sprint nên chưa tách được burndown riêng cho mỗi Sprint.

4 Phân công và commit

Thuận commit dưới bốn tên Git: Thuan-Test, Trần Vĩnh Thuận, Thuan-Lap và Thuan-AFK666. Toàn bộ dữ liệu Git nằm ở `git-evidence.txt`.

Thành viên	Commit, tác giả	Nhánh	Nội dung
Bảo Tín, Task 1	`44e3784`, Saturlen	`main`	README, Docker Compose, Dockerfile, mã backend
Phú, Task 2	`a358930`, `5d92e1f`, phu0111	`a1task2-9`	Đặc tả API danh sách sản phẩm
Bảo Thiện, Task 3	`9ea0933`, baothien-s	`thien35`, merge `862bf65`	Trang Menu, gọi API, trạng thái tải và rỗng
Lợi, Task 4	`32831dc`, DuVinhLoi	`feat/task4-task7`, merge `eaf33f8`	Chi tiết món, trạng thái tải và lỗi, test
Bảo Thiện, Task 5	`9ea0933`, baothien-s	`thien35`, merge `862bf65`	Giỏ hàng, tổng tiền, badge
Bảo Tín, Task 6	`44e3784`, Saturlen	`main`	Controller, DTO và bảng giá của API tạo đơn
Lợi, Task 7	`32831dc`, DuVinhLoi	`feat/task4-task7`, merge `eaf33f8`	JWT, chuẩn hóa email, test xác thực
Thuận, Task 8	`bd1adc3`, Thuan-Test	`feature/task-8-10-payment-order`, merge `03cb553`	Thanh toán qua cổng giả lập, chuyển trạng thái sau thanh toán
Phú, Task 9	`ed2e4ee`, phu0111	`a1task2-9`	Đặc tả API xác nhận đơn và lịch sử đơn
Thuận, Task 10	`bd1adc3`, Thuan-Test	`feature/task-8-10-payment-order`, merge `03cb553`	Máy trạng thái, idempotency, tồn kho, điểm thưởng

5 Minh chứng cho 10 task

Mỗi task gồm tiêu chí của đề, các ca kiểm thử trong collection Postman, file mã nguồn, test tự động và ảnh chụp. Request trong collection đặt tên theo mã ca, ví dụ `T06.5` là ca thứ 5 của Task 6. Chi tiết từng ca nằm ở `docs/minh-chung-10-task.md`.

Ảnh chụp đặt vào thư mục `figures/` cạnh `main.tex`, đúng tên file ghi trong khung. Khung ghi "Chưa có ảnh" là ảnh nhóm chưa chụp.

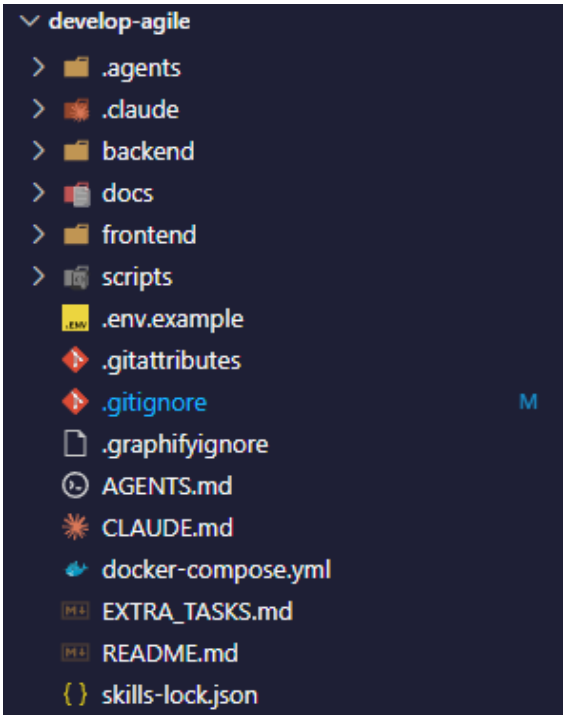
5.1 Task 1 — Khởi tạo dự án và cấu trúc

Tiêu chí của đề. Repo có thư mục `frontend` chạy Next.js và `backend` chạy NestJS, chạy được trang chào, có README, Git và `.env.example`.

Mã ca	Kiểm tra	Kết quả mong đợi
T01.1	`GET {{webUrl}}/`	200, HTML có chữ "BrewLite"
T01.2	`GET {{baseUrl}}/products`	200, JSON

Mã nguồn. `README.md`, `.env.example`, `frontend/package.json`, `backend/package.json`, `docker-compose.yml`.

Test. `backend/test/bootstrap-env.spec.ts` kiểm backend dừng khi thiếu `DATABASE_URL`; `backend/test/task47-auth.spec.ts` kiểm backend dừng khi thiếu `JWT_SECRET`.



Hình 2: Minh chứng Task 1: cây thư mục repo

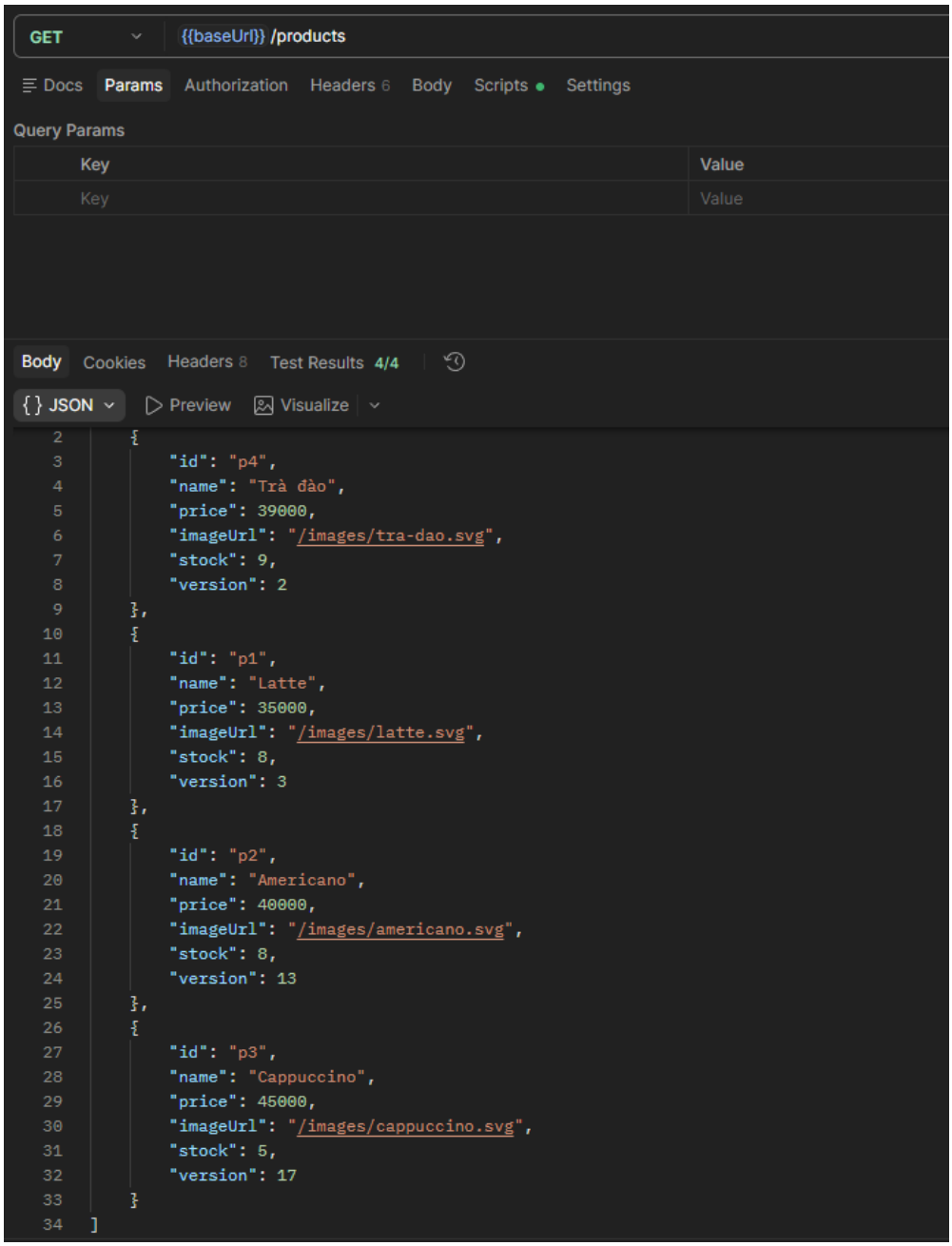
5.2 Task 2 — API danh sách sản phẩm

Tiêu chí của đề. `GET /products` trả JSON menu gồm id, tên, giá, ảnh.

Mã ca	Kiểm tra	Kết quả mong đợi
T02.1	`GET /products`	Mảng món, mỗi món có `id`, `name`, `price`, `imageUrl`; không có món `stock = 0`
T02.2	`GET /products/:id` với id có thật	Đúng món đó
T02.3	`GET /products/:id` với id không có	404

Mã nguồn. `backend/src/product/product.controller.ts`, dữ liệu mẫu `backend/prisma/seed.ts`.

Test. `backend/test/order-product.spec.ts`, ca "product list excludes stock-0" và "product detail unknown → NotFound".



Hình 3: Postman ca T02.1: tab Body và Test Results

5.3 Task 3 — Trang Menu

Tiêu chí của đề. Next.js gọi `/products`, hiện lưới sản phẩm, có trạng thái đang tải và danh sách rỗng.

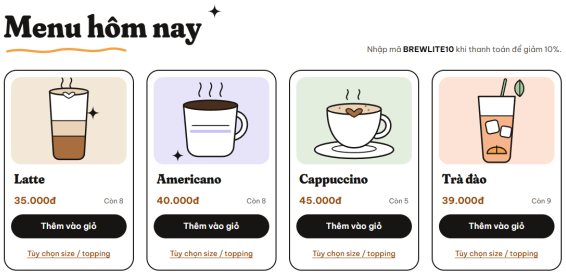
Mã ca	Kiểm tra	Kết quả mong đợi
T03.1	`GET {{webUrl}}/`	200
T03.2	Tải ảnh theo `imageUrl` của một món	200, kiểu `image/svg+xml`

Postman chỉ kiểm được phía server. Phần hiển thị dựa vào test vitest và ảnh chụp trình

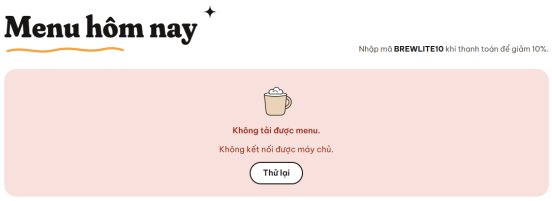
duyệt.

Mã nguồn. `frontend/src/components/ProductGrid.tsx`, hàm `getProducts` trong `frontend/src/lib/api.ts`.

Test. `frontend/src/components/ProductGrid.test.ts` kiểm ba trạng thái đang tải, rỗng và lỗi kèm nút "Thử lại"; `frontend/src/lib/api.test.ts`.



(a) Lưới món tại `http://localhost:3001`



(b) Lỗi khi tắt backend bằng `docker compose stop backend`

Hình 4: Minh chứng Task 3

5.4 Task 4 — Chi tiết và tùy chọn sản phẩm

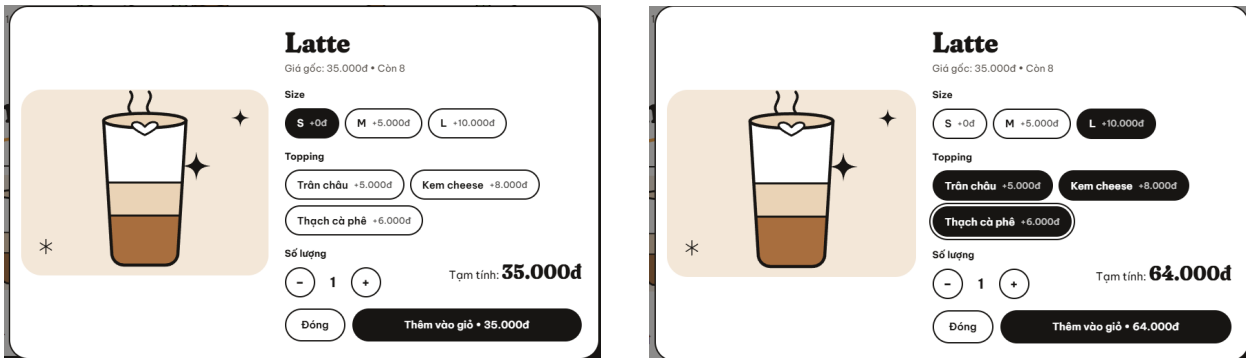
Tiêu chí của đề. Chọn size S, M, L và topping; giá đổi theo tùy chọn.

Size M cộng 5.000 đồng, size L cộng 10.000 đồng. Topping Trân châu giá 5.000 đồng, Kem cheese giá 8.000 đồng. Frontend và backend dùng cùng bảng giá này.

Mã ca	Kiểm tra	Kết quả mong đợi
T04.1	Đặt size L, topping Trân châu và Kem cheese	`unitPrice` = giá gốc + 10.000 + 13.000
T04.2	Đặt size S, không topping	`unitPrice` = giá gốc
T04.3	Đặt với topping không có trong bảng giá	400 `Unknown topping`

Mã nguồn. `frontend/src/components/ProductDetailModal.tsx`, `frontend/src/lib/pricing.ts`, `backend/src/order/pricing.ts`.

Test. `frontend/src/lib/pricing.test.ts`, `frontend/src/components/ProductDetailModal.test.ts`, `frontend/src/lib/product-detail.test.ts`; `backend/test/pricing-table.spec.ts` kiểm bảng giá hai phía trùng nhau.



(a) Trước khi đổi size và topping (b) Sau khi đổi, giá tăng tương ứng

Hình 5: Modal chi tiết món

5.5 Task 5 — Giỏ hàng

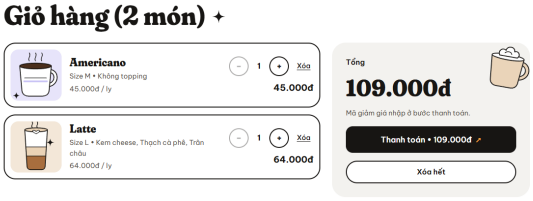
Tiêu chí của đề. Thêm, sửa số lượng, xóa; lưu state; tự tính tổng; badge số lượng.

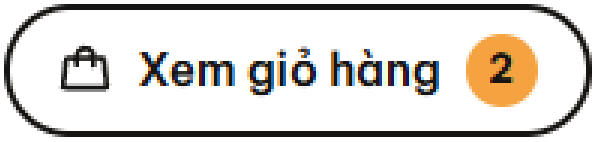
Giỏ giới hạn 20 dòng và 1 đến 99 ly mỗi dòng; backend kiểm lại cùng giới hạn khi tạo đơn.

Mã ca	Kiểm tra	Kết quả mong đợi
T05.1	Tạo đơn từ giỏ 2 dòng	`total` = tổng `quantity` × `unitPrice`
T05.2	Tạo đơn từ giỏ 21 dòng	400
T05.3	Tạo đơn có một dòng 100 ly	400

Mã nguồn. Store Zustand `frontend/src/store/cart.ts`, `frontend/src/components/cart/CartView.tsx`, badge trong `frontend/src/components/HeaderView.tsx`.

Test. `frontend/src/store/cart.test.ts`, `frontend/src/lib/cart-limits.test.ts`, `frontend/src/components/cart/CartView.test.ts`, `frontend/src/components/QtyStepper.test.ts`, `frontend/src/components/HeaderView.test.ts`.





(a) Giỏ có 2 món và tổng tiền (b) Badge số lượng trên header

Hình 6: Minh chứng Task 5

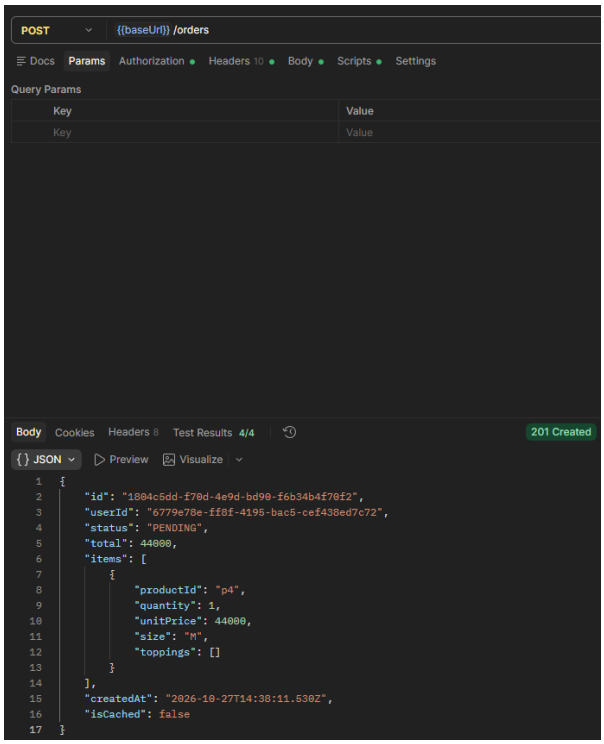
5.6 Task 6 — API tạo đơn hàng

Tiêu chí của đề. `POST /orders` nhận giỏ hàng, validate bằng class-validator, lưu đơn ở trạng thái Chờ thanh toán và trả mã đơn.

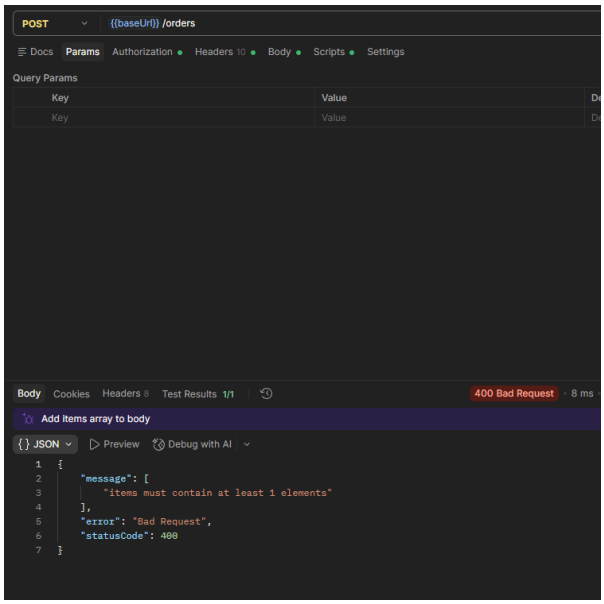
Mã ca	Kiểm tra	Kết quả mong đợi
T06.1	Gửi không kèm token	401
T06.2	Giỏ rỗng	400
T06.3	Size `XL`	400
T06.4	Món không có trong menu	404
T06.5	Đơn hợp lệ, cố tình gửi kèm `unitPrice: 1`	201, `status` = `PENDING`, có `id`; giá do server tính, bỏ qua giá 1 đồng
T06.6	Gửi bằng token nhân viên quầy	403

Mã nguồn. `backend/src/order/order.controller.ts`, `backend/src/order/dto/create-order.dto.ts`, `backend/src/order/pricing.ts`.

Test. `backend/test/order-product.spec.ts`, `backend/test/pricing-table.spec.ts`, `backend/test/customer-only.spec.ts`.



(a) Ca T06.5: Body có `id` và `status`



(b) Ca T06.2: 400 kèm thông báo validate

Hình 7: Minh chứng Task 6

5.7 Task 7 — Đăng ký và đăng nhập bằng JWT

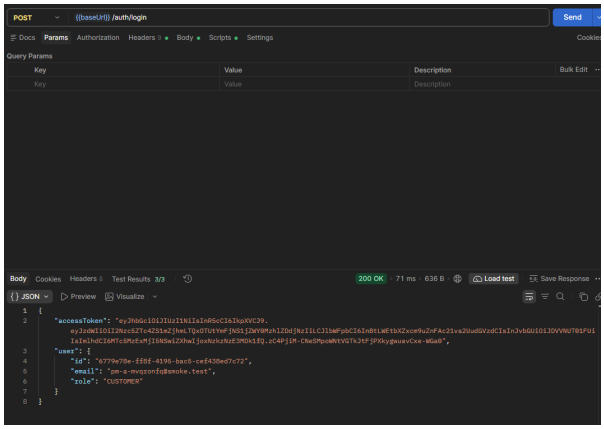
Tiêu chí của đề. `POST /auth/register` và `POST /auth/login`; băm mật khẩu bằng bcrypt; phát JWT; guard bảo vệ route đặt đơn.

Mã ca	Kiểm tra	Kết quả mong đợi
T07.1	Đăng ký email mới	201, body không có `passwordHash`
T07.2	Đăng ký lại cùng email viết hoa	409
T07.3	Email sai định dạng, mật khẩu quá ngắn	400
T07.4	Đăng nhập đúng	JWT đủ 3 phần, `role` = `CUSTOMER`
T07.5	Đăng nhập sai mật khẩu	401 `Invalid credentials`
T07.6	Đăng nhập email không tồn tại	401 `Invalid credentials`
T07.7	Gọi API bằng token giả	401

Ca sai mật khẩu và ca email không tồn tại trả cùng một thông báo, nên người ngoài không dò được email nào đã đăng ký. Guard của route đặt đơn được kiểm ở ca T06.1.

Mã nguồn. `backend/src/auth/auth.controller.ts`, `backend/src/auth/auth.service.ts`, `backend/src/auth/jwt-auth.guard.ts`, `frontend/src/app/login/page.tsx`.

Test. `backend/test/auth.spec.ts`, `backend/test/task47-auth.spec.ts`, `backend/test/roles.spec.ts`, `frontend/src/lib/login.test.ts`, `frontend/src/lib/session.test.ts`, `frontend/src/components/LoginView.test.ts`.



(a) Ca T07.4: có `accessToken`

```
      email       |  left
------------------+--------
 new@brewlite.vn  | $2b$10$
 demo@brewlite.vn | $2b$10$
 barista@brewlite.vn | $2b$10$
(3 rows)
```

(b) Bảng `User`: `passwordHash` bắt đầu bằng `$2b$`, tức chuỗi băm bcrypt

Hình 8: Minh chứng Task 7

Lệnh lấy ảnh (b):

```
docker compose exec db psql -U brewlite \
  -c 'select email, left("passwordHash", 7) from "User" limit 3;'
```

5.8 Task 8 — Thanh toán không tiền mặt

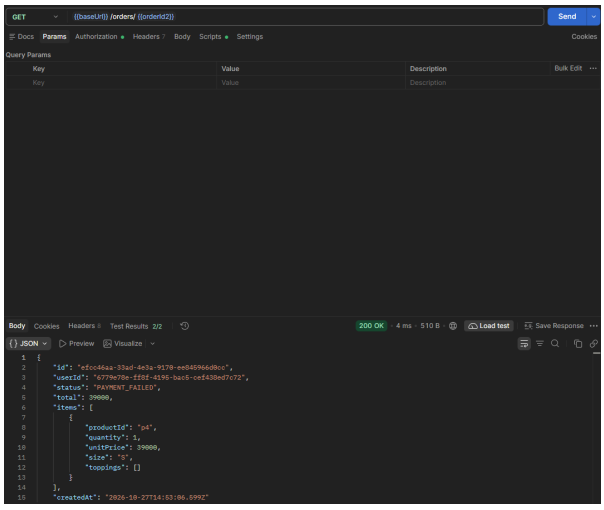
Tiêu chí của đề. Chọn Ví hoặc Thẻ, gọi `POST /payments`. Thanh toán thành công thì đơn sang Đã thanh toán; lỗi thì đơn sang Thanh toán lỗi và giỏ giữ nguyên.

Để tái hiện luồng thanh toán lỗi, nhóm dựng stack với biến môi trường `MOCK_GATEWAY_DECLINE_METHOD=CARD`. Biến này buộc cổng giả lập từ chối mọi giao dịch bằng Thẻ, còn Ví vẫn thành công.

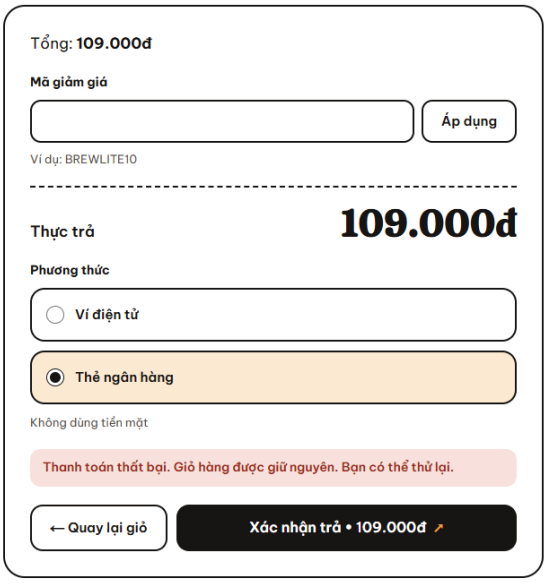
Mã ca	Kiểm tra	Kết quả mong đợi
T08.1	Trả bằng Ví	201, `PAID`, có `paymentId`
T08.2	Đọc lại đơn vừa trả	`status` = `PAID`
T08.3–T08.5	Trả bằng Thẻ, cổng giả lập từ chối	400 `Payment failed`; đơn sang `PAYMENT_FAILED`
T08.6	Trả lại bằng Ví trên đơn vừa lỗi	201, `PAID`, cùng mã đơn cũ
T08.7	Thiếu header `Idempotency-Key`	400
T08.8	Phương thức `CASH`	400

Mã nguồn. `backend/src/payment/payment.controller.ts`, `backend/src/payment/payment.service.ts`, `backend/src/payment/mock-payment-gateway.service.ts`, `frontend/src/components/PaymentForm.tsx`.

Test. `backend/test/payment-failure.spec.ts`, `backend/test/order-payment.integration.spec.ts`, `frontend/src/components/PaymentForm.test.ts`, `frontend/src/lib/payment/api-gateway.test.ts`, `frontend/src/lib/payment/checkout-session.test.ts`.



(a) Ca T08.5: đơn ở `PAYMENT_FAILED`



(b) Màn thanh toán báo lỗi khi chọn Thẻ, giỏ còn nguyên

Hình 9: Minh chứng Task 8

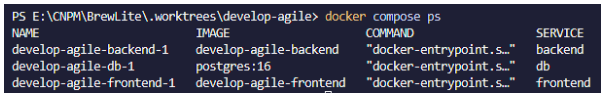
5.9 Task 9 — Xác nhận, lịch sử đơn và bàn giao

Tiêu chí của đề. Màn xác nhận có mã đơn và trạng thái; `GET /orders/me` trả lịch sử đơn; Docker Compose chạy đủ frontend, backend và cơ sở dữ liệu; có README; demo được trọn luồng.

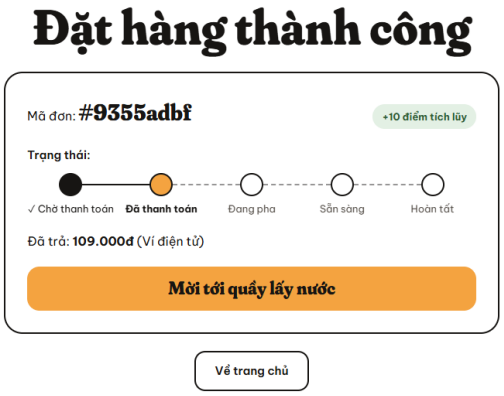
Mã ca	Kiểm tra	Kết quả mong đợi
T09.1	`GET /orders/:id` đơn của mình	Có `id`, `status`, `total`, `items`
T09.2	`GET /orders/me`	Có các đơn vừa tạo, mới nhất trước, mỗi dòng có `productName`
T09.3	Khách B gọi `GET /orders/me`	Không thấy đơn của khách A
T09.4	Khách B đọc đơn của khách A	403
T09.5	Mã đơn không tồn tại	404
T09.6	`GET {{webUrl}}/history`	200

Mã nguồn. `backend/src/order/order.controller.ts`, `frontend/src/app/history/page.tsx`, `frontend/src/components/PaymentSuccess.tsx`, `docker-compose.yml`, `README.md`.

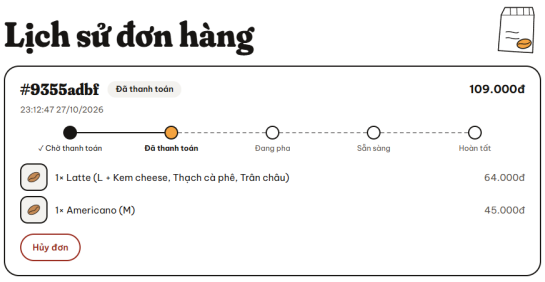
Test. `backend/test/order-product.spec.ts`, ca "listMine returns newest-first"; `backend/test/compose-env.spec.ts`; `frontend/src/components/HistoryView.test.ts`, `frontend/src/components/PaymentSuccess.test.ts`, `frontend/src/lib/history.test.ts`, `frontend/src/lib/use-order-poll.test.ts`.



(a) `docker compose ps` có `db`, `backend`, `frontend`



(b) Biên lai có mã đơn, trạng thái Đã thanh toán



(c) Trang `/history`

Hình 10: Minh chứng Task 9

5.10 Task 10 — Nghiệp vụ backend

Tiêu chí của đề. Mục 9 của đề yêu cầu đủ bốn phần: máy trạng thái đơn hàng, thanh toán idempotent, kiểm soát tồn kho khi đặt đồng thời, khuyến mãi và điểm thưởng. Test phải

chứng minh ba điều: chặn được chuyển trạng thái sai; gửi hai lần cùng `Idempotency-Key` chỉ tạo một đơn; đặt đồng thời không vượt tồn kho.

(a) Máy trạng thái đơn hàng. Hàm `assertTransition` trong `backend/src/state-machine/order-state.machine.ts` chặn mọi chuyển tiếp không có trong bảng dưới với mã 400. Hoàn tất và Đã hủy là trạng thái cuối. Quyền được kiểm riêng trước khi kiểm chuyển tiếp: khách gọi API tiến đơn bị chặn với mã 403 dù chuyển tiếp đó hợp lệ.

Từ	Sang	Bên thực hiện
`PENDING`	`PAID`	Backend, khi cổng giả lập chấp nhận giao dịch
`PENDING`	`PAYMENT_FAILED`	Backend, khi cổng giả lập từ chối
`PAYMENT_FAILED`	`PENDING`	Backend, khi khách thanh toán lại đơn đó
`PAID`	`PREPARING`	Barista hoặc Admin
`PREPARING`	`READY`	Barista hoặc Admin
`READY`	`COMPLETED`	Barista hoặc Admin
`PENDING`	`CANCELLED`	Khách chủ đơn
`PAYMENT_FAILED`	`CANCELLED`	Khách chủ đơn
`PAID`	`CANCELLED`	Khách chủ đơn, hoặc Barista và Admin khi hết nguyên liệu

Giao diện hiển thị các trạng thái theo nhãn trong `frontend/src/lib/order-status.ts`: `PENDING` là Chờ thanh toán, `PAID` là Đã thanh toán, `PREPARING` là Đang pha, `READY` là Sẵn sàng, `COMPLETED` là Hoàn tất, `PAYMENT_FAILED` là Thanh toán lỗi, `CANCELLED` là Đã hủy.

Mã ca	Kiểm tra	Kết quả mong đợi
T10a.1	Khách tự tiến đơn của mình	403 `Only staff can advance orders`
T10a.2	Quầy chuyển thẳng `PAID` sang `READY`	400 `Invalid transition PAID -> READY`
T10a.3	Quầy tiến đơn sang `PREPARING`	200
T10a.4	Khách hủy đơn đang `PREPARING`	400
T10a.5, T10a.6	Quầy tiến đơn sang `READY`, rồi `COMPLETED`	200 mỗi bước
T10a.7	Hủy đơn đã `COMPLETED`	400 `Invalid transition COMPLETED -> CANCELLED`
T10a.8	Đọc lại đơn	Vẫn `COMPLETED`
T10a.9	Quầy gọi `GET /orders?status=PAID`	Có đơn vừa thanh toán
T10a.10	Quầy hủy đơn `PAID`	200, `CANCELLED`

Mã nguồn. `backend/src/state-machine/order-state.machine.ts`; hàm `advanceOrder` và `cancelOrderAs` trong `backend/src/payment/payment.service.ts`.

Test. `backend/src/state-machine/order-state.machine.spec.ts`, `backend/test/order-advance.spec.ts`, `backend/test/barista-queue.spec.ts`, `frontend/src/lib/order-status.test.ts`.

(b) Thanh toán idempotent. Backend lưu `Idempotency-Key` với ràng buộc duy nhất ở bảng Order và bảng Payment. Request lặp lại cùng khóa nhận lại kết quả cũ với mã 200, không tạo đơn mới, không trừ tiền, không trừ kho và không cộng điểm thêm lần nữa.

Mã ca	Kiểm tra	Kết quả mong đợi
T10b.2, T10b.3	`POST /orders` hai lần cùng khóa	Lần 1 trả 201; lần 2 trả 200, cùng `id`, `isCached: true`
T10b.5, T10b.6	`POST /payments` hai lần cùng khóa	Lần 2 trả 200, cùng `paymentId`
T10b.7	Đọc điểm thưởng	Chỉ cộng một lần
T10b.8	Đọc tồn kho	Chỉ trừ một lần, đúng 2 ly
T10b.9	Khách khác dùng lại khóa đó	422

Test. `backend/test/order-payment.integration.spec.ts`, ca "(b) same Idempotency-Key charges once".

(c) Tồn kho khi đặt đồng thời. Kho bị trừ lúc thanh toán, trong cùng transaction với việc chuyển đơn sang Đã thanh toán, ghi điểm thưởng và lưu biên lai. Mỗi lần trừ kho so cột `Product.version`: bản ghi đã bị request khác sửa thì lần ghi thất bại và backend đọc lại, thử tối đa 3 lần. Cách khóa này gọi là optimistic locking. Hết 3 lần vẫn không trừ được thì transaction rollback, backend hoàn tiền và đơn sang Thanh toán lỗi.

Mã ca	Kiểm tra	Kết quả mong đợi
T10c.1	Admin ghi kho với `version` cũ	409 `Stock changed, reload`
T10c.2	Admin trừ quá số còn lại	400 `Stock cannot be negative`
T10c.3	Khách gọi API kho	403

Postman Runner gửi request lần lượt nên không tạo được tải đồng thời. Trường hợp nhiều khách đặt cùng lúc được chứng minh bằng test "(c) concurrent stock never negative" trong `backend/test/order-payment.integration.spec.ts` và `backend/test/pg-payment.integration.spec.ts`. Test thứ hai chạy trên PostgreSQL thật và chỉ chạy khi có biến `DATABASE_URL`; lần chạy ngày 21/10/2026 không đặt biến này nên 2 ca của nó bị bỏ qua.

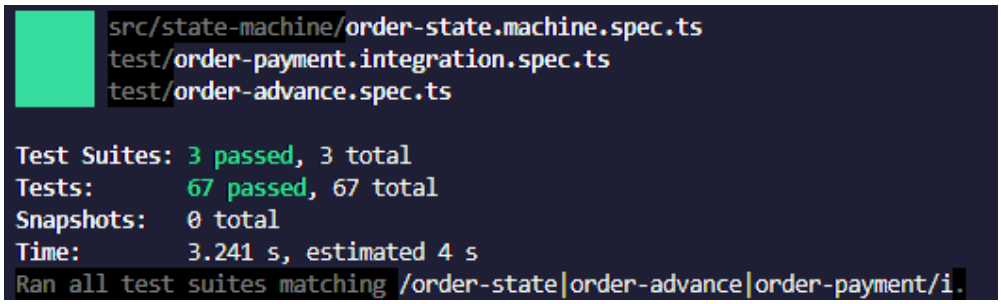
(d) Khuyến mãi và điểm thưởng. Đề không quy định mã và tỉ lệ, nhóm chọn mã `BREWLITE10` giảm 10%:

$$\text{giảm giá} = \lfloor \text{tổng} \times 0{,}1 \rfloor, \qquad \text{số tiền trả} = \text{tổng} - \text{giảm giá}, \qquad \text{điểm} = \left\lfloor \frac{\text{số tiền trả} \times 0{,}1}{1000} \right\rfloor.$$

Điểm chỉ cộng sau khi đơn sang Đã thanh toán. Lần chạy ngày 21/10/2026: đơn 58.000 đồng được giảm 5.800 đồng, khách trả 52.200 đồng và được cộng 5 điểm.

Mã ca	Kiểm tra	Kết quả mong đợi
T10d.3	Trả kèm mã `BREWLITE10`	`discount`, `amount`, `loyaltyPointsEarned` đúng công thức
T10d.4	`GET /users/me`	Thấy điểm vừa cộng
T10d.6	Trả kèm mã không hợp lệ `SAIMA`	`discount` = 0

Test. `backend/test/order-payment.integration.spec.ts`, các ca của mã `BREWLITE10`.



Hình 11: Minh chứng Task 10: `npx jest order-state order-advance order-payment` chạy xanh

6 Kiểm thử, Definition of Done và bàn giao

6.1 Kết quả kiểm thử ngày 21/10/2026

Bộ kiểm thử	Kết quả
Postman, 74 request	137 assertion đạt, 0 lỗi; phản hồi trung bình 14 ms, lớn nhất 108 ms
Backend, `npx jest`	170 test đạt, 3 test bỏ qua, 173 tổng
Frontend, `npx vitest run`	733 test đạt trong 46 file

Ba test backend bị bỏ qua đều cần PostgreSQL thật qua biến `DATABASE_URL`: 2 ca idempotency và tồn kho đồng thời trong `pg-payment.integration.spec.ts`, 1 ca dọn dữ liệu thử trong `pg-smoke-clean.integration.spec.ts`. Log Postman lưu ở `docs/postman/results/2026-10-21-run.txt`.

6.2 Definition of Done

Điều kiện (mục 10 của đề)	File xác nhận

Code chạy, không lỗi build	`docker-compose.yml`
	`backend/Dockerfile`
	`frontend/Dockerfile`

Đúng tiêu chí chấp nhận của task	`docs/postman/BrewLite.postman_collection.json`
	`docs/postman/results/2026-10-21-run.txt`
	`docs/minh-chung-10-task.md`

Có commit rõ ràng, được review trong nhóm	`git-evidence.txt`

API validate đầu vào, UI xử lý đang tải và lỗi	`backend/src/order/dto/create-order.dto.ts`
	`frontend/src/components/ProductGrid.tsx`
	`frontend/src/components/ProductGrid.test.ts`

README có hướng dẫn chạy	`README.md` `.env.example`

6.3 Kịch bản demo bàn giao

1. Dựng ba dịch vụ theo README, chụp `docker compose ps`.
2. Đăng ký, đăng nhập, xem menu, đổi size và topping, xem giá đổi.
3. Thêm hai món, sửa số lượng, xóa một món, xem badge và tổng tiền.
4. Thanh toán bằng Ví, xem biên lai có mã đơn và trang lịch sử.
5. Thanh toán bằng Thẻ khi cổng giả lập từ chối, xem giỏ còn nguyên, rồi thanh toán lại bằng Ví trên cùng đơn.
6. Chạy thư mục Task 10 trong Postman: tiến đơn bằng tài khoản quầy, gửi lặp cùng khóa, ghi kho với `version` cũ.
7. Chạy test tồn kho đồng thời trên PostgreSQL thật.
8. Bàn giao mã nguồn, `.env.example`, README, collection Postman và log kiểm thử; ghi phản hồi của khách hàng vào Product Backlog.

6.4 Sprint Retrospective

Điều làm tốt.

- Nghiệp vụ khó đều có test tự động: idempotency (gửi lặp cùng khóa vẫn chỉ tạo một đơn), máy trạng thái và tồn kho khi đặt đồng thời.
- Bảo mật được làm ở backend, không phụ thuộc giao diện: server tự tính giá và bỏ qua giá client gửi lên (ca T06.5), mật khẩu băm bằng bcrypt.
- Xong đủ 10 task trước hạn.

Điều chưa làm tốt.

- Không làm theo thứ tự Sprint, và commit dồn thành đợt.
- Review trong nhóm còn yếu.
- Không ghi lại các sự kiện Scrum.

7 Nguồn và cách chạy lại

- `docs/BrewLite-CongNghePhanMem-fixed.md`: đề, tiêu chí 10 task và Definition of Done.
- `docs/minh-chung-10-task.md`: chi tiết từng ca Postman, test và ảnh cần chụp.
- `docs/product-backlog.md`: 10 user story và tiêu chí chấp nhận.
- `docs/postman/BrewLite.postman_collection.json` và `docs/postman/results/2026-10-21-run.txt`: collection và log Postman.
- `git-evidence.txt`: commit của từng thành viên.

Chạy lại kiểm thử từ thư mục gốc của repo:

```
MOCK_GATEWAY_DECLINE_METHOD=CARD docker compose up -d --build
postman collection run docs/postman/BrewLite.postman_collection.json
(cd backend && npx jest)
(cd frontend && npx vitest run)
```